

DATA 260 Homework 5

Student: Vrishin KP

Project: Package Vulnerability Reporting Platform

Technology: FastAPI, SQLAlchemy, MySQL, React, Redux Toolkit, Axios, and Vite

1. Objective

The main goal of this homework was to improve the existing vulnerability reporting application by connecting the React frontend to Redux Toolkit. In the earlier version, some report information and CRUD actions were handled directly inside individual pages. For this homework, I moved the main report state and report operations into Redux so that the application has a more organized and predictable data flow.

This change also helped connect the frontend and backend more clearly. The React pages collect user input, Redux manages the application state, Axios sends the request, and FastAPI performs the database operation. I also added the new HW5 fields required by the backend, including the package ID and affected versions count.

2. Implemented Features

I implemented the following features and checked each one through the running application:

- Connected the report list to the Redux store.
- Added Redux thunks for loading, creating, updating, and deleting reports.
- Added package ID support to the create and update forms.
- Added affected-version count support.
- Updated the backend update function to save affected-version counts.
- Updated the backend timestamp when a report is modified.
- Connected the delete screen to the Redux delete thunk.
- Verified the production frontend build using Vite.

One important part of the work was making sure that the update form did not only display the new fields but also sent them to the backend. During testing, I found that the affected-version count was still being saved as zero. I corrected the backend update function and then verified that the value was saved and displayed correctly.

3. Redux Data Flow

Redux is now used as the central place for the report list and report operations. When a user performs an action, the page calls a handler in `App.jsx`. That handler dispatches a Redux thunk. The thunk calls the Axios API function, and the result is stored back in the Redux state.

This structure keeps the page components focused on displaying forms and collecting input. It also means that the report list can be updated after creating, editing, or deleting a report without keeping separate copies of the same data in multiple components.

Operation	Frontend flow	Backend result
Load	<code>fetchReports()</code>	GET vulnerability reports
Create	<code>addReport(payload)</code>	POST a new vulnerability report
Update	<code>editReport({ id, payload })</code>	PUT updated report data
Delete	<code>removeReport(id)</code>	DELETE the selected report

The final request path is: page component → App.jsx → Redux thunk → Axios API function → FastAPI endpoint → database.

4. Verification Results

I tested the application through the browser using the local React frontend and FastAPI backend. The tests were performed after logging in with the demo account. The results below show what happened during each test.

Test	Result	Evidence
User login and report loading	Passed	The authenticated Home page opened and displayed vulnerability reports.
Update report	Passed	The affected versions count was changed to 5 and displayed correctly after the update.
Create report	Passed	A new report with a unique vulnerability ID appeared in the report list.

Delete report	Passed	The selected report was deleted and no longer appeared on the Home page.
Production build	Passed	Vite completed successfully with 95 modules transformed.

5. Test Evidence

The following screenshots document the main tests performed in the browser. Sensitive information such as passwords and authentication tokens was not included in the screenshots.

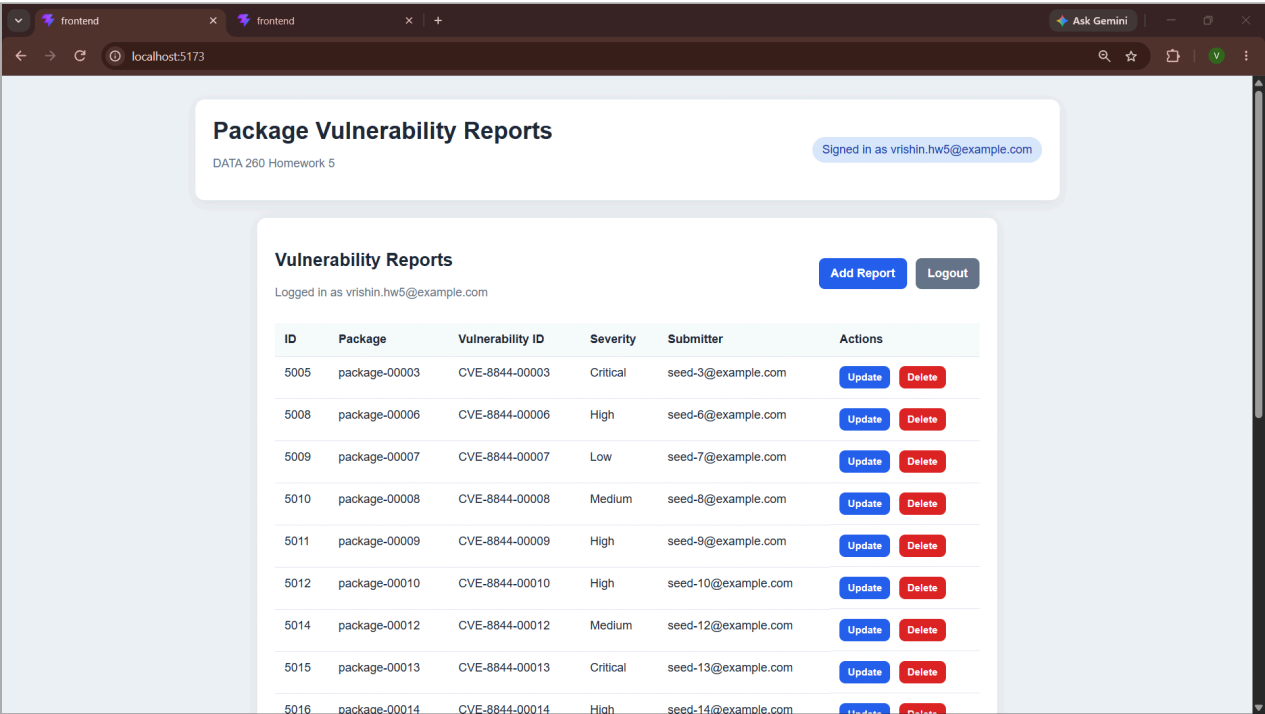


Figure 1. Authenticated Home page showing the vulnerability reports.

Update Vulnerability Report

Package name
package-00006

Package ID
6

Vulnerability ID
CVE-8844-00006

Submitter email
seed-6@example.com

Vulnerability description
Deterministic benchmark vulnerability description for record 6.

Severity
High

Affected versions count
5

☒ I accept the terms and conditions.

Update Vulnerability Report

Figure 2. Update form showing the affected versions count set to 5.

Package Vulnerability Reports

DATA 260 Homework 5

Signed in as vishin.hw5@example.com

Vulnerability Reports

Logged in as vishin.hw5@example.com

Add Report Logout

ID	Package	Vulnerability ID	Severity	Submitter	Actions
5005	package-00003	CVE-2026-8844-TEST-003	Critical	seed-3@example.com	Update Delete
5008	package-00006	CVE-8844-00006	High	seed-6@example.com	Update Delete
5009	package-00007	CVE-8844-00007	Low	seed-7@example.com	Update Delete
5010	package-00008	CVE-8844-00008	Medium	seed-8@example.com	Update Delete
5011	package-00009	CVE-8844-00009	High	seed-9@example.com	Update Delete
5012	package-00010	CVE-8844-00010	High	seed-10@example.com	Update Delete
5014	package-00012	CVE-8844-00012	Medium	seed-12@example.com	Update Delete
5015	package-00013	CVE-8844-00013	Critical	seed-13@example.com	Update Delete
5016	package-00014	CVE-8844-00014	High	seed-14@example.com	Update Delete

Figure 3. Home page showing the newly created vulnerability report.

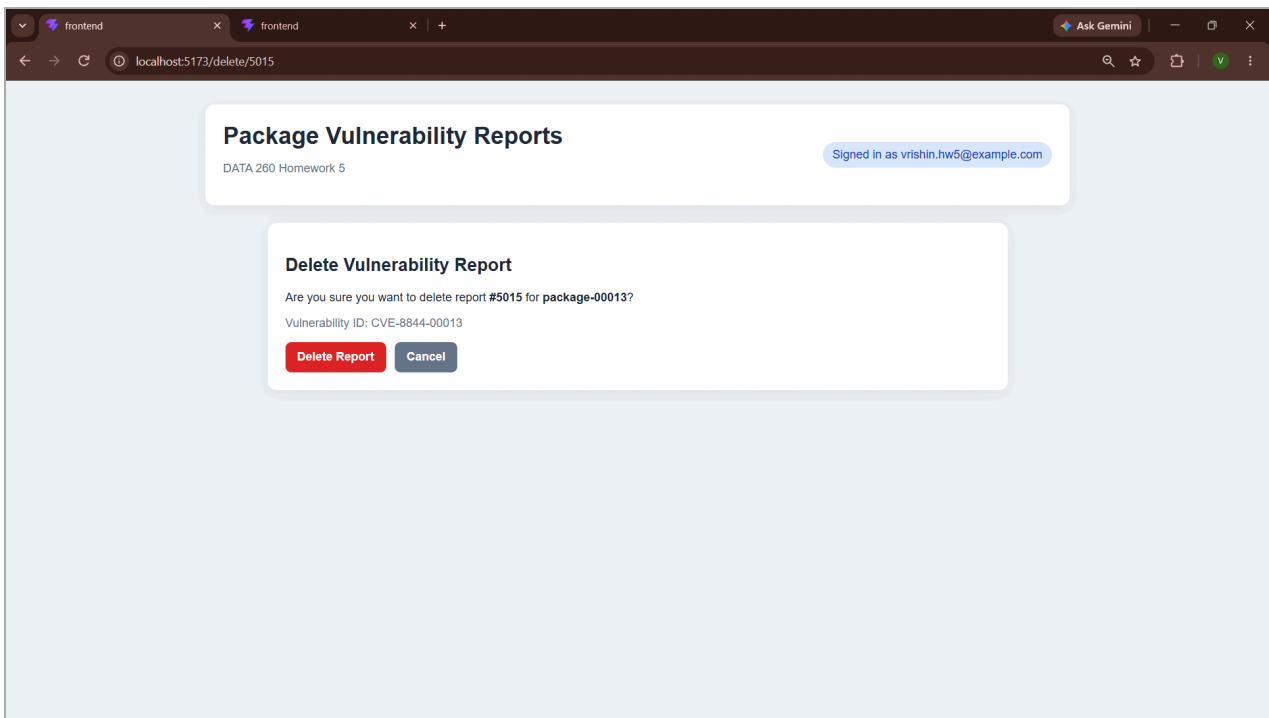


Figure 4. Delete confirmation page before removing the report.

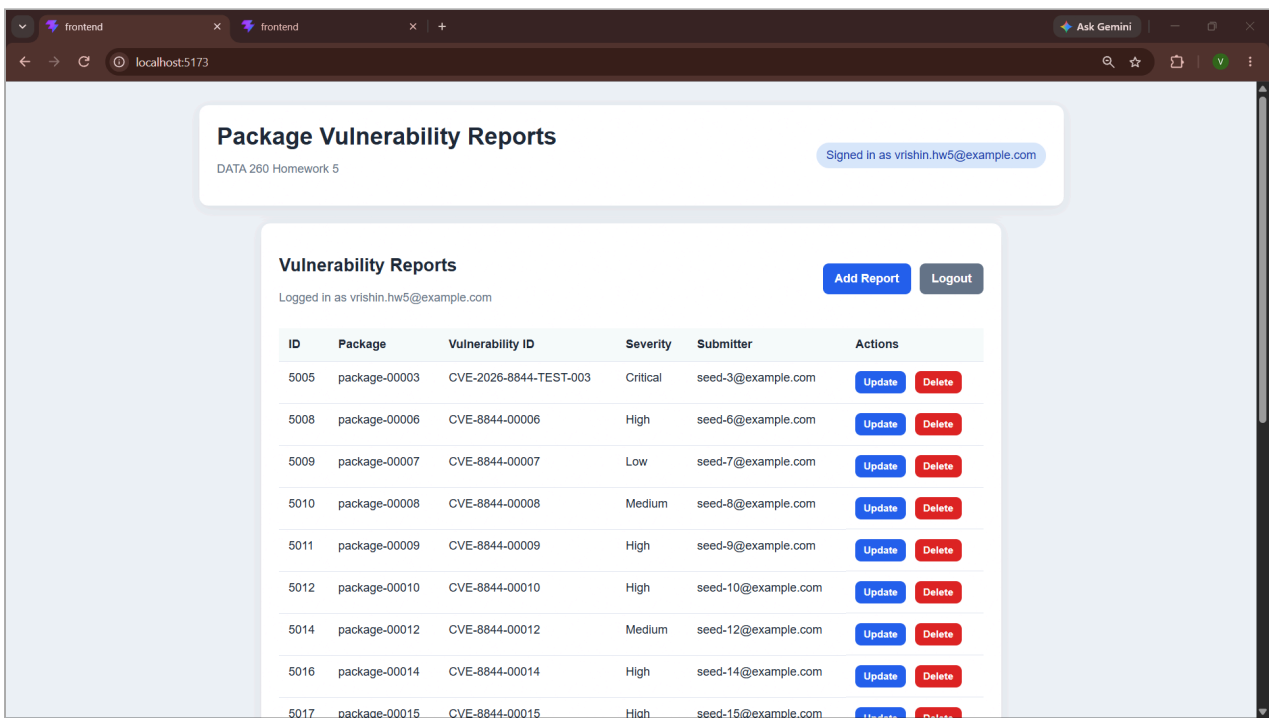


Figure 5. Home page after deletion, showing that the report was removed.

```

PS C:\Projects\data260-8844> cd "C:\Projects\data260-8844\frontend"
PS C:\Projects\data260-8844\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.1 building client environment for production...
/ 95 modules transformed.
computing gzip size...
dist/index.html          0.45 kB | gzip: 0.29 kB
dist/assets/index-wprW19rh.css 1.95 kB | gzip: 0.85 kB
dist/assets/index-B1Vd4o0J.js 349.11 kB | gzip: 112.62 kB

/ built in 240ms

```

Figure 6. Successful production build using `npm run build`.

6. Important Project Files

The following files were important to the HW5 implementation. Each file contributed to a different part of the frontend or backend workflow.

- `frontend/src/App.jsx` - connects the application pages to Redux actions and selectors.
- `frontend/src/features/reports/reportsSlice.js` - stores report state and defines the asynchronous Redux thunks.
- `frontend/src/store/store.js` - configures the Redux store for the React application.
- `frontend/src/pages/CreateRecord.jsx` - collects the fields needed to create a vulnerability report.
- `frontend/src/pages/UpdateRecord.jsx` - loads an existing report and submits updated values.
- `frontend/src/pages/DeleteRecord.jsx` - displays the delete confirmation and calls the Redux delete handler.
- `code/web_application/crud_hw4.py` - performs the database create, read, update, and delete operations.
- `code/web_application/schemas.py` - validates the request and response data used by the API.

7. Challenges and Fixes

The main issue found during testing was that the affected versions count appeared as zero after updating a report. At first, the frontend form was sending the value, but the backend update function was not assigning the value to the database object.

I corrected the backend update logic so that it saves `affected_versions_count` and updates the report timestamp. After restarting the backend, I repeated the browser test and confirmed that the value 5 was saved and displayed correctly.

Another small issue occurred while editing the delete page. The file was accidentally opened from a nested frontend path. I moved the corrected file into the actual `frontend/src/pages` directory, removed the accidental duplicate folder, rebuilt the frontend, and confirmed that the build still passed.

9. Submission Information

This submission was prepared on the HW5 development branch. The final version will be identified in Git using the annotated submission tag `hw5` .

Repository	Viki2183/data260-8844
Branch	hw5-development
Submission tag	hw5
Report location	reports/hw05/

8. Conclusion

Homework 5 was completed by integrating Redux Toolkit into the existing vulnerability reporting application. The report list and CRUD operations now follow a consistent Redux-based data flow.

The browser tests confirmed that a user can log in, view reports, create a new report, update an existing report, save the affected versions count, and delete a report. The final Vite production build also completed successfully, confirming that the frontend code is valid and can be bundled for deployment.